

Yunliang CAO GROUP 3(seule)

Guide de fonctionnement du projet :

Au début, vous pouvez utiliser la commande :

make

Cela générera une bibliothèque partagée dans le répertoire « library » et un fichier exécutable « codec » dans le répertoire actuel. Si vous souhaitez générer ces éléments séparément, utilisez les commandes suivantes :

make library
make codec

Pour générer des fichiers doxygen, utilisez :

make doxygen

Si vous oubliez les commandes ci-dessus, tapez :

make help

Après avoir généré le fichier « codec », vous pouvez exécuter la commande suivante :

./codec -h

Ceci affichera une interface d'aide, comme suit :

```

cao@cyl:/mnt/e/L3/C/Projet2$ ./codec -h
Usage: ./codec [option] -c|-u -i <input> -o <output> -g -a <alpha>
  -c          encode, input PGM, output QTC
  -u          decode, input QTC, output PGM
  -i <file>   input filename (.pgm or .qtc)
  -o <file>   output filename (default out.qtc or out.pgm)
  -a <alpha>  filtrage
  -g          grille
  -h          help

Example:
./codec -c -i ./PGM/input.pgm -o ./QTC/output.qtc -a 1.5
./codec -u -i ./QTC/input.qtc -o ./PGM/output.pgm

```

Points importants :

1. Si vous souhaitez personnaliser le nom du fichier de sortie, ajoutez le chemin à son nom, par exemple : `./QTC` ou `./PGM`. Sinon, le fichier sera directement généré dans le répertoire actuel. Il en va de même pour les fichiers d'entrée à spécifier avec `-i`.
2. Vous pouvez décider d'activer la grille (option `-g`) lors de l'encodage ou du décodage. Cependant, si la grille est activée pendant l'encodage, elle sera obligatoirement présente lors du décodage.
- 3..L'option `-a` peut être spécifiée pendant le décodage, mais elle n'aura aucun effet.
4. Si vous utilisez à la fois `-c` et `-u`, seule la dernière option sera prise en compte. Par exemple :

```

cao@cyl:/mnt/e/L3/C/Projet2$ ./codec -c -u -i ./QTC/out.qtc
Finished decoding => ./PGM/out.pgm

```

Cela n'exécutera que `-u`

Réflexions sur le projet :

Ce projet semble avoir répondu à toutes les exigences, y compris l'encodage, le décodage, la compression sans perte et avec perte (je pense que tout est correct, mais je ne peux pas garantir qu'il soit exempt de bugs). En règle générale, le programme fonctionne.

Difficultés rencontrées :

Au cours de ce projet, j'ai rencontré de nombreux problèmes. Cependant, comme j'ai commencé à rédiger ce rapport dix jours après avoir terminé l'ensemble du projet (entre-temps, j'ai dû travailler sur du Java), j'ai oublié beaucoup de ces difficultés ; je ne peux donc aborder ici que celles dont je me souviens.

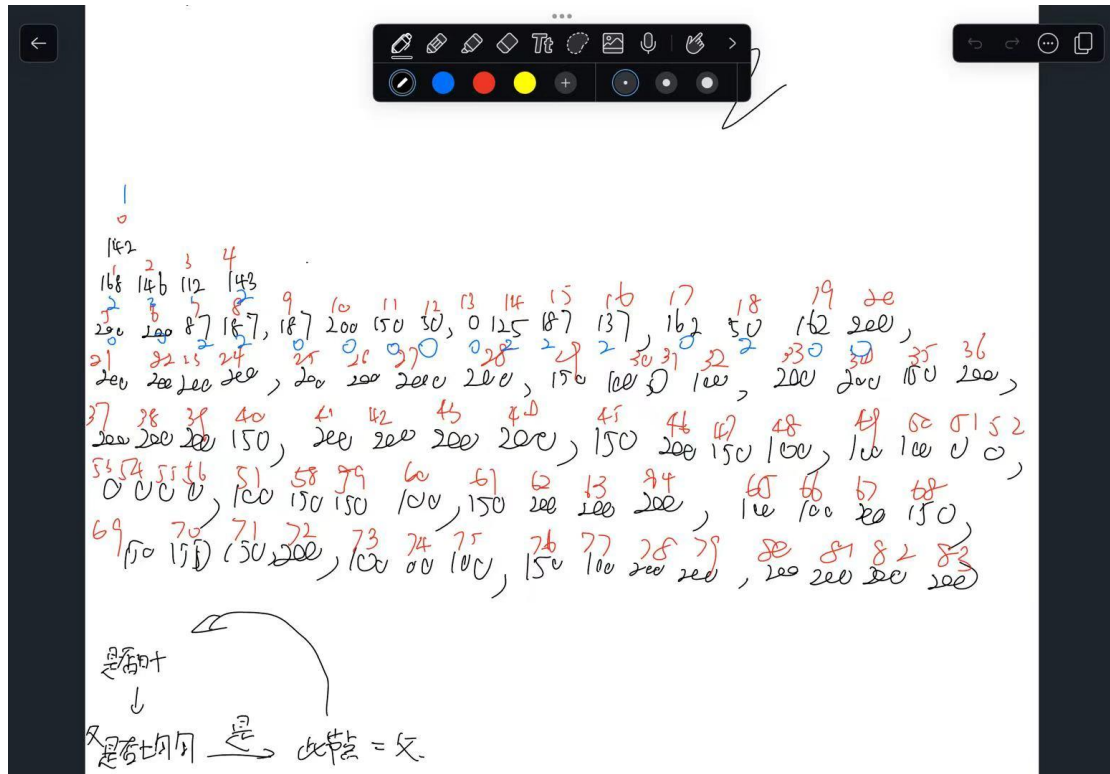
Pour commencer, la difficulté majeure a été de comprendre intégralement ce PDF de 12 pages : ce n'était pas simplement une question de le traduire dans une langue que je maîtrise, mais aussi d'en saisir les principes sous-jacents. Comme je n'avais pas bien assimilé ce document au début, j'ai eu d'énormes difficultés dès la première étape de la construction de l'arbre. Quand j'ai finalement réussi à construire l'arbre à l'aide de pointeurs, j'ai relu le document et découvert qu'il fallait utiliser un bloc de mémoire contigu (autrement dit un tableau) pour stocker cet arbre. J'ai donc tout supprimé et tout recommencé. Ensuite, j'ai eu des ennuis avec la récursivité : le tableau généré n'était pas ordonné par niveaux, mais plutôt avec chaque nœud enfant suivi immédiatement de son nœud parent. Ce problème m'a longtemps perturbé, et ce n'est qu'au bout d'une journée environ que j'ai réalisé que la logique de la récursivité était erronée. En décalant l'appel récursif d'une ligne, tout a enfin fonctionné.

Lorsque j'ai voulu stocker l'arbre dans un fichier, je n'avais pas compris qu'il fallait travailler bit par bit. J'ai d'abord enregistré les nombres codés de façon brute, et cela ne correspondait pas du tout à l'exemple fourni. En relisant la documentation, j'ai compris qu'il fallait écrire dans le fichier au niveau du bit. J'ai alors réfléchi au fait que je devrais aussi relire ces mêmes données plus tard à partir d'un fichier qtc. J'ai donc créé une structure dédiée spécifiquement à ces opérations sur les bits (écriture, lecture, extension de capacité, etc.), dont le but principal était de maîtriser la logique de la manipulation au niveau du bit.

La phase de décompression a d'abord semblé se dérouler sans problème, mais une fois terminée, j'ai constaté un important décalage sur toute une colonne à gauche. Après vérification et mise en commentaire ligne par ligne, j'ai découvert que ce problème venait de la ligne de commentaire “#” : celle-ci ne doit pas précéder les données elles-mêmes, mais doit être placée juste avant les deux nombres indiquant la taille de l'image. Je ne sais pas exactement pourquoi, mais cela a résolu le problème.

J'ai rencontré un autre souci : mon image décompressée comportait de nombreux petits points blancs ou noirs, des pixels très réduits. Comme je n'en comprenais pas l'origine, j'ai dû chercher manuellement. Le plus contraignant, c'est que pour une image 4x4, tout fonctionnait bien, mais plus l'image était grande, plus les problèmes étaient nombreux. J'ai donc pris une image 8x8 et procédé à des impressions, copies et comparaisons ligne par ligne. Finalement, j'ai découvert que

j'avais bel et bien extrait la valeur e, mais que je ne l'avais jamais assignée, ce qui entraînait une accumulation d'erreurs et provoquait ces décalages. J'ai gardé cette image en guise de souvenir et, heureusement, j'ai trouvé le bug au vingtième pixel ; sinon, je ne sais pas combien de temps j'aurais encore dû y consacrer.



Enfin, la partie concernant le **filtrage** a été relativement plus simple. La seule difficulté, pendant un court moment, a été de comprendre comment obtenir l'effet de séparation similaire à celui de l'exemple. Finalement, il s'agissait simplement de changer la couleur des pixels autour des blocs où $u=1$. Comme les dimensions de chaque bloc étaient déjà transmises, il suffisait d'utiliser une formule pour identifier directement les pixels autour de ces blocs. Avec quatre boucles, ce problème a pu être résolu facilement.